

Aplicações Web

Professor Lozano

Tópicos que iremos abordar na aula:

- Configuração do ambiente de desenvolvimento (instalação do JDK, IntelliJ IDEA, Docker Desktop, Node.js, Visual Studio Code, Postman e Git);
- Criação do projeto da API com o Spring Initializr;
- Versionamento do projeto com Git e GitHub (repositório remoto, clone, commit e push);
- Abertura e configuração do projeto no IntelliJ IDEA, incluindo a resolução de problemas comuns de configuração.

Aplicações Web

Professor Lozano

Passo 1 – Instalação do JDK

Toda aplicação Java precisa de um ambiente preparado para rodar, e é exatamente esse o papel do JDK (Java Development Kit). Ele é o kit de ferramentas que vai nos dar tudo que precisamos: compilador, runtime e as bibliotecas essenciais para desenvolver.

Nesta aula, vamos instalar o JDK versão 25. E por que essa versão específica? Porque ela é uma versão LTS (Long-Term Support), ou seja, uma versão com suporte estendido, pensada para ambientes de produção. A recomendação de mercado é sempre trabalhar com a LTS mais recente disponível, e a versão 25 tem suporte garantido pela Oracle até setembro de 2028, o que nos dá tranquilidade para desenvolver sem preocupação com fim de suporte no meio do caminho.

Para instalar, acesse o site oficial da Oracle e escolha a versão LTS mais recente disponível para o seu sistema operacional, no nosso caso, Windows:

<https://www.oracle.com/br/java/technologies/downloads/#jdk25-windows>

Java 26, Java 25, Java 21, and earlier versions available now

JDK 26 is the latest release of the Java SE Platform.

JDK 25 is the latest *Long-Term Support (LTS)* release of the Java SE Platform.

JDK 21 is the previous Long-Term Support (LTS) release of the Java SE Platform.

Earlier JDK versions are available below.

[Learn about Java SE Subscription](#)

JDK 26 JDK 25 JDK 21

Java SE Development Kit 25.0.3 downloads

JDK 25 binaries are free to use in production and free to redistribute, at no cost, under the [Oracle No-Fee Terms and Conditions \(NFTC\)](#).

JDK 25 will receive updates under the NFTC, until September 2028, a year after the release of the next LTS. Subsequent JDK 25 updates will be licensed under the [Java SE OTN License \(OTN\)](#) and production use beyond the [limited free grants](#) of the OTN license will [require a fee](#).

Linux macOS Windows

Product/file description	File size	Download
x64 Compressed Archive	205.71 MB	https://download.oracle.com/java/25/latest/jdk-25_windows-x64_bin.zip (sha256)
x64 Installer	184.51 MB	https://download.oracle.com/java/25/latest/jdk-25_windows-x64_bin.exe (sha256)
x64 MSI Installer	183.28 MB	https://download.oracle.com/java/25/latest/jdk-25_windows-x64_bin.msi (sha256)

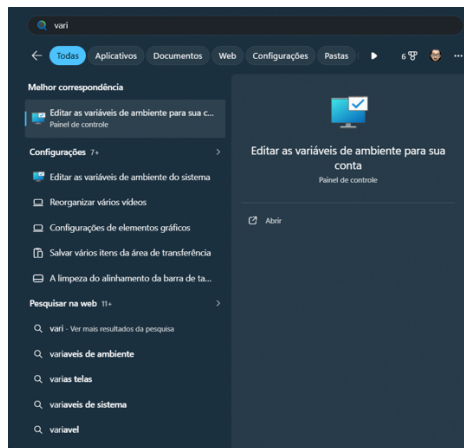
Com o download concluído, é hora de executar o instalador. Essa primeira parte é bem tranquila: basta ir clicando em NEXT até a instalação ser finalizada, o instalador cuida da maior parte do trabalho pesado pra você.

Só que instalar o JDK sozinho não é o suficiente. Precisamos agora configurar as variáveis de ambiente, para que o sistema operacional saiba onde encontrar o Java e consiga executá-lo a partir de qualquer lugar (por exemplo, direto pelo terminal).

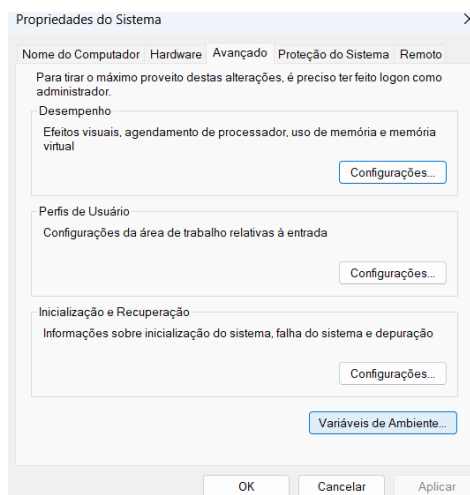
Aplicações Web

Professor Lozano

Para isso, clique em Buscar (ou Pesquisar, dependendo da versão do Windows) e digite "variáveis":



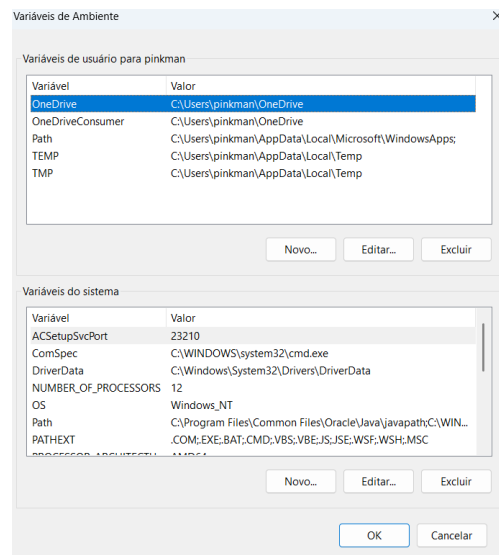
Acesse a opção "Editar as variáveis de ambiente do sistema". A seguinte tela será aberta:



Aplicações Web

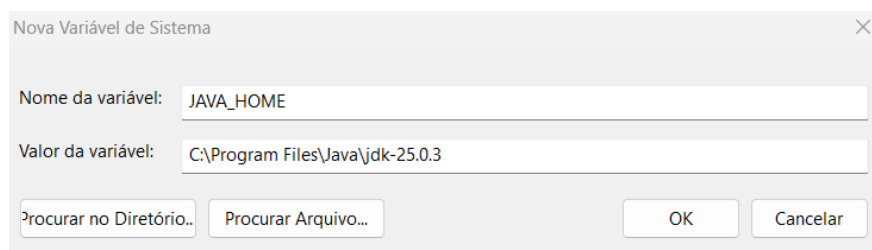
Professor Lozano

Clique na opção "Variáveis de Ambiente...". A seguinte tela será aberta:



Agora vamos trabalhar na seção "Variáveis do sistema", são elas que valem para todos os usuários do sistema operacional (diferente das variáveis de usuário, que afetam só o seu perfil de login). Como queremos que o Java funcione para qualquer usuário da máquina, é aqui que vamos mexer.

Clique em Novo e preencha com as seguintes configurações:



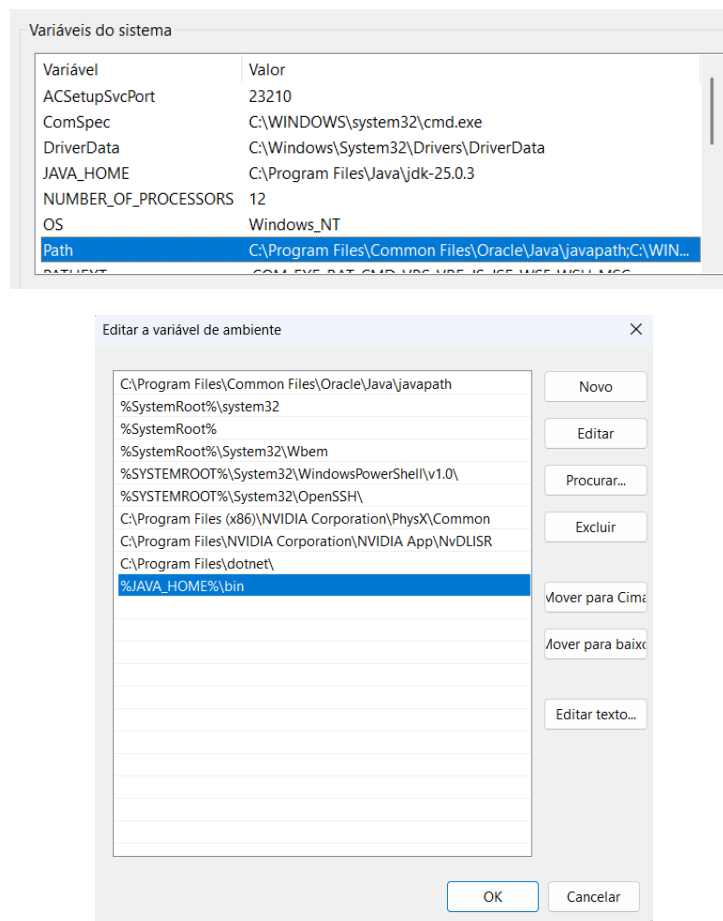
Obs.: o valor dessa variável deve apontar para a raiz da pasta onde o JDK foi instalado (geralmente algo como C:\Program Files\Java\jdk-25).

Com o JAVA_HOME criado, vamos agora ajustar a variável Path. Ela é a responsável por dizer ao Windows onde procurar os executáveis quando você digita um comando no terminal, sem isso, o sistema não vai saber o que fazer quando você chamar o java pela linha de comando.

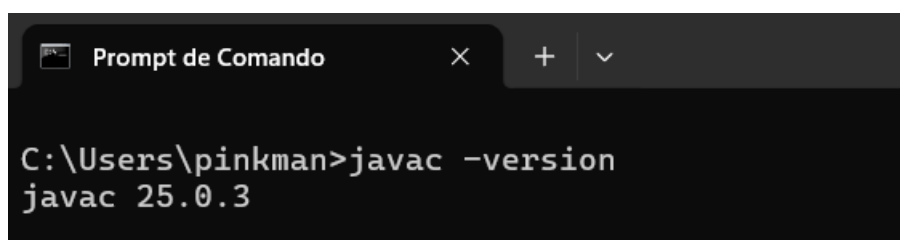
Aplicações Web

Professor Lozano

Localize a variável Path, clique em Editar e adicione uma nova entrada com o seguinte valor: %JAVA_HOME%\bin



Com tudo configurado, chegou a hora de validar se deu certo. Vamos abrir o Prompt de Comando e digitar: `javac -version`. Esse comando é o nosso "teste de fogo": se as variáveis de ambiente foram configuradas corretamente, ele deve retornar a versão do compilador Java instalada, confirmando que o sistema já reconhece o JDK e está pronto para compilarmos nossos programas.



Aplicações Web

Professor Lozano

Obs.: o javac é um programa que fica justamente dentro da pasta bin do JDK, aquele mesmo caminho que adicionamos na variável Path. Ou seja, ao configurar as variáveis de ambiente, o que fizemos na prática foi liberar os recursos do Java para serem utilizados pelo nosso sistema operacional, permitindo que qualquer terminal, em qualquer pasta, consiga reconhecer e executar os comandos do Java.

Passo 2 – Instalação do IntelliJ

Uma das ferramentas que vamos usar ao longo da disciplina é o IntelliJ IDEA, uma das IDEs mais populares do mercado quando o assunto é desenvolvimento Java.

Até meados de 2025, o IntelliJ IDEA era distribuído em duas versões separadas: a Ultimate (paga, com recursos avançados) e a Community (gratuita, com o essencial para programar em Java). No final de 2025, a JetBrains decidiu unificar tudo em um único produto, hoje você baixa apenas "IntelliJ IDEA", sem precisar escolher entre edições diferentes.

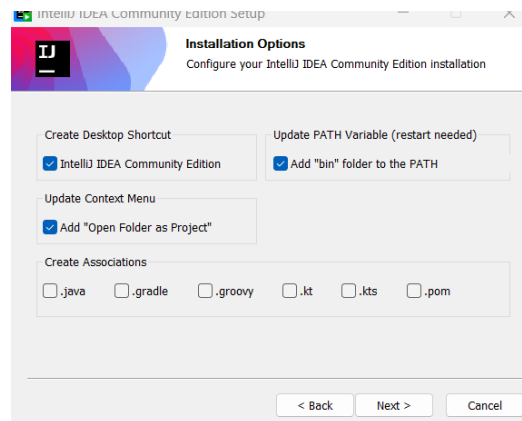
Para a nossa disciplina, vamos utilizar a versão gratuita disponível. Acesse o site oficial, clique em BAIXAR, selecione a opção "Other Downloads" e escolha a opção desejada:



Aplicações Web

Professor Lozano

Com o download concluído, vamos executar o instalador. A instalação segue o padrão que já vimos antes, avance clicando em Next até chegar na seguinte tela, na qual você não precisa alterar nada, apenas mantenha as opções como estão por padrão:

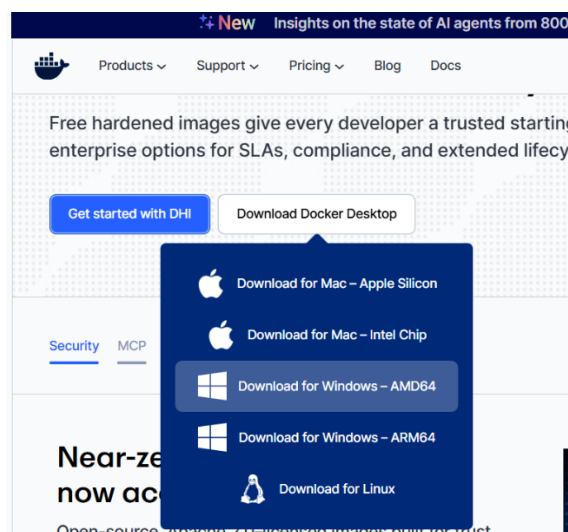


Na sequência, é só continuar avançando (Next) normalmente até a instalação ser concluída, sem necessidade de nenhuma configuração extra nessa etapa.

Passo 3 – Instalação do Docker

Outra ferramenta essencial para o nosso curso é o Docker. Ele vai nos ajudar em várias frentes ao longo da disciplina: subir bancos de dados em containers, realizar o deploy da nossa aplicação e muito mais, sem a dor de cabeça de instalar e configurar cada dependência manualmente na máquina.

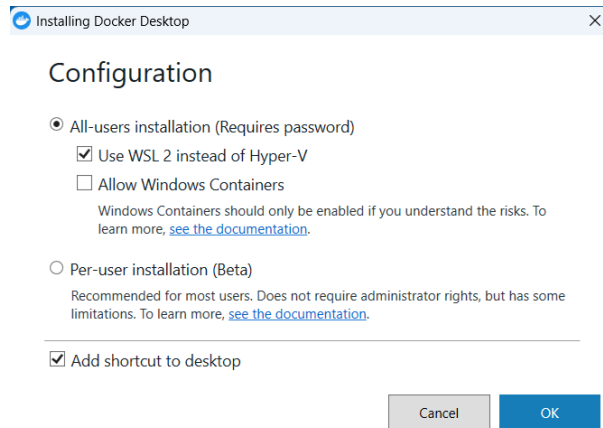
Para instalar, acesse www.docker.com e faça o download do Docker Desktop:



Aplicações Web

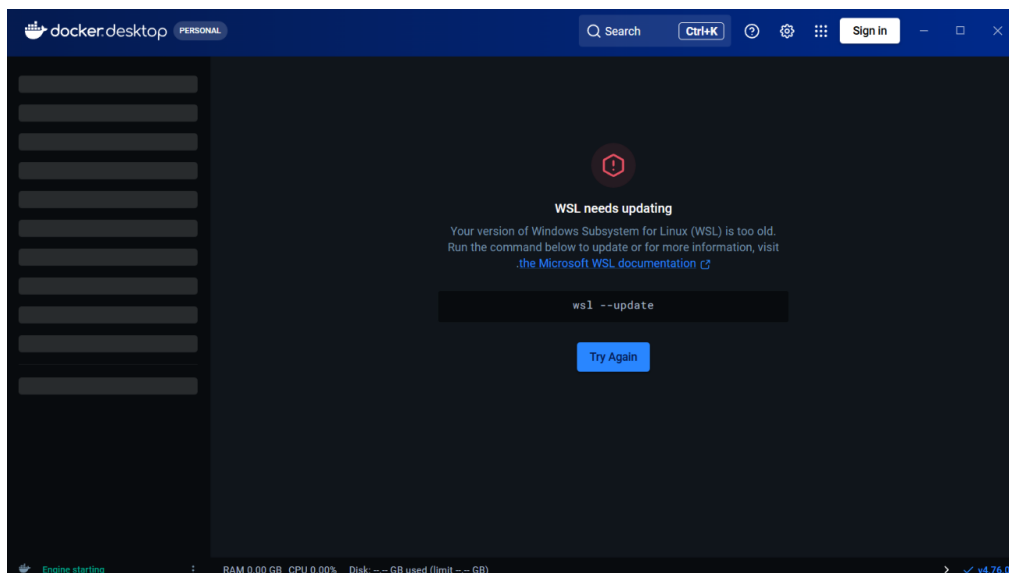
Professor Lozano

Após concluir o download, clique para executar o arquivo de instalação. Durante esse processo, existe apenas uma opção que precisamos ficar de olho, o restante pode seguir no padrão:



Depois dessa etapa, é só continuar avançando até finalizar a instalação. Na sequência, vamos abrir o Docker Desktop.

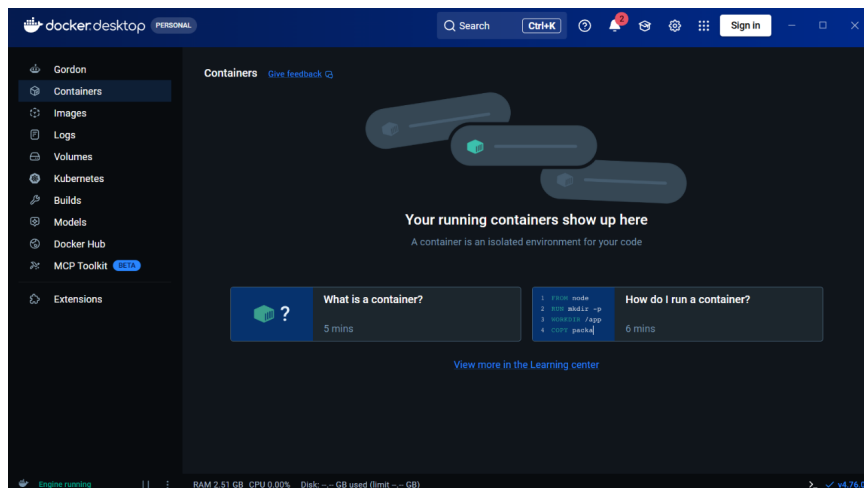
É bem comum que, nesse momento, o Windows solicite uma atualização do WSL (Windows Subsystem for Linux), afinal, o Docker Desktop depende dele para rodar os containers Linux por baixo dos panos, mesmo estando no Windows. Caso essa mensagem apareça, precisamos abrir o Prompt de Comando e executar o seguinte comando: `wsl --update`.



Após executar o comando no Prompt de Comando, o WSL será atualizado. Ao concluir, inicie o Docker Desktop novamente. Se tudo correu bem, a seguinte tela deve aparecer:

Aplicações Web

Professor Lozano

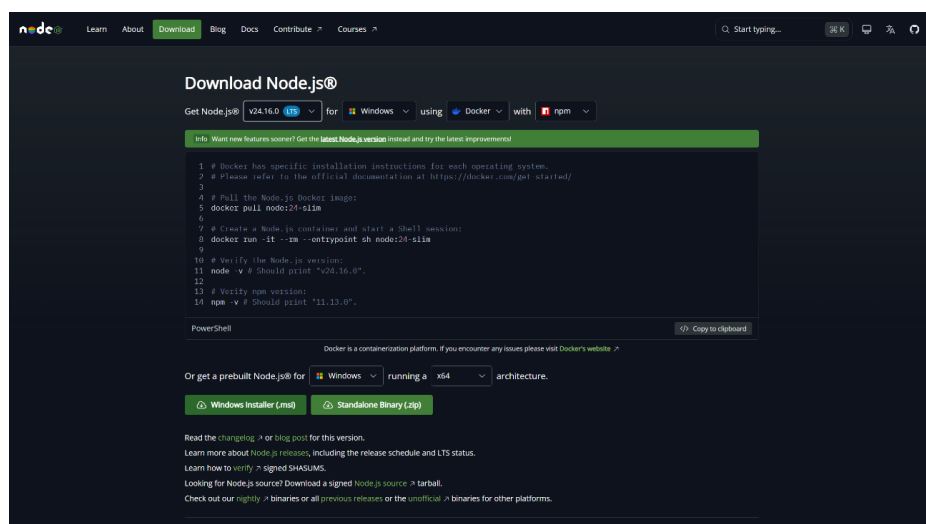


Passo 4 – Instalação do NodeJS.

Mais uma ferramenta importante para o nosso curso é o Node.js. Acesse <https://nodejs.org/en> e faça o download da versão LTS mais atual disponível.

Vale reforçar o conceito: assim como fizemos com o JDK, aqui também priorizamos a versão LTS (Long-Term Support), ou seja, a versão com o maior tempo de suporte oficial, ideal para ambientes de desenvolvimento e produção que exigem estabilidade.

Para o nosso ambiente Windows, vamos baixar o arquivo no formato .msi:



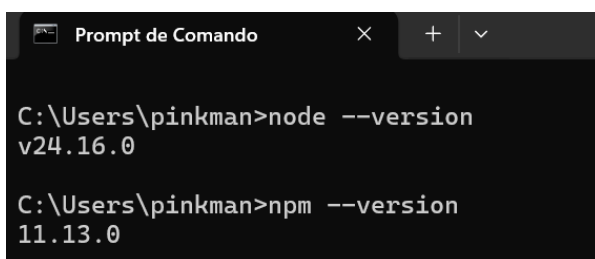
Aplicações Web

Professor Lozano

Para instalar o Node.js, basta executar o arquivo de instalação e avançar em todas as opções, não é necessário alterar nenhuma configuração durante o processo.

Após concluir, vamos abrir o Prompt de Comando e validar a instalação digitando os seguintes comandos: `node --version` e `npm --version`.

Se tudo correu bem, cada comando deve retornar a versão instalada, confirmando que o Node.js e o npm (gerenciador de pacotes que já vem junto com o Node) estão prontos para uso.



```
C:\Users\pinkman>node --version
v24.16.0

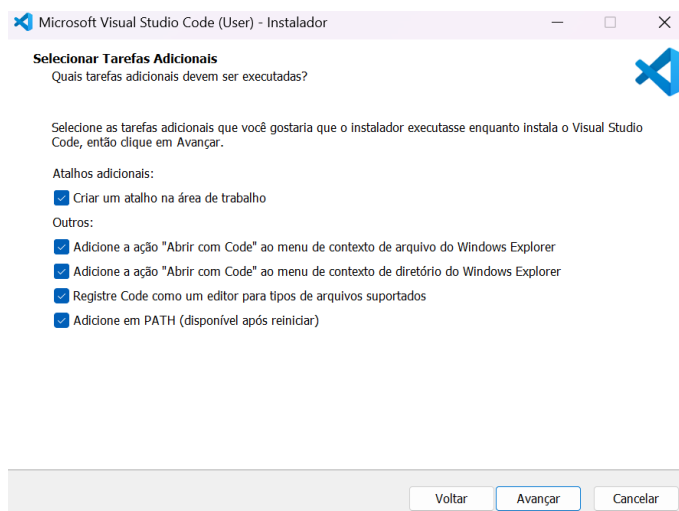
C:\Users\pinkman>npm --version
11.13.0
```

Passo 5 – Instalação do VSCode.

Para trabalharmos com o React, vamos precisar do Visual Studio Code. Ele começou como um editor de texto, mas evoluiu tanto ao longo dos anos, com suporte a uma infinidade de plugins e extensões, que atualmente já é considerado uma verdadeira IDE por boa parte da comunidade de desenvolvedores.

Acesse <https://code.visualstudio.com/> e faça o download da versão compatível com o seu sistema operacional.

Para instalar, execute o arquivo baixado e avance até chegar na seguinte opção:



Após selecionar as opções, avance normalmente até concluir a instalação.

Aplicações Web

Professor Lozano

Passo 6 - Instalação do Postman.

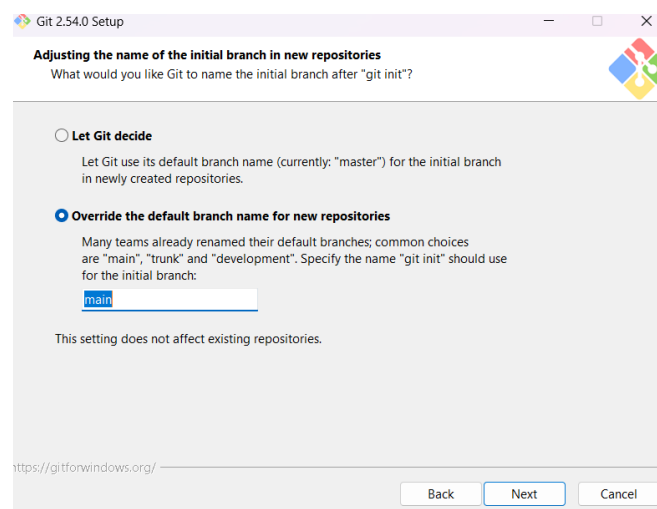
Outra ferramenta que vamos precisar é o Postman. Ao longo do curso, quando começarmos a desenvolver nossa API, ele será essencial para testarmos as rotas, ou seja, simular requisições HTTP (GET, POST, PUT, DELETE etc.) e conferir se nossa aplicação está respondendo do jeito esperado, sem precisar já ter um front-end pronto pra isso.

Para instalar, acesse <https://www.postman.com/downloads/> e faça o download da versão compatível com o seu sistema operacional. Após concluir o download, basta executar o arquivo para realizar a instalação, sem etapas extras ou configurações especiais nesse processo.

Passo 7 – Instalação do Git.

Para fechar nossa lista de ferramentas, vamos instalar o Git. Ele será nosso controle de versão ao longo de todo o curso, essencial para acompanhar o histórico de mudanças do código, colaborar em equipe e, claro, integrar com o GitHub (ou outra plataforma) quando chegarmos na etapa de deploy.

Acesse <https://git-scm.com/install/windows> e baixe o instalador para Windows. Execute o arquivo e avance até chegar na seguinte configuração:



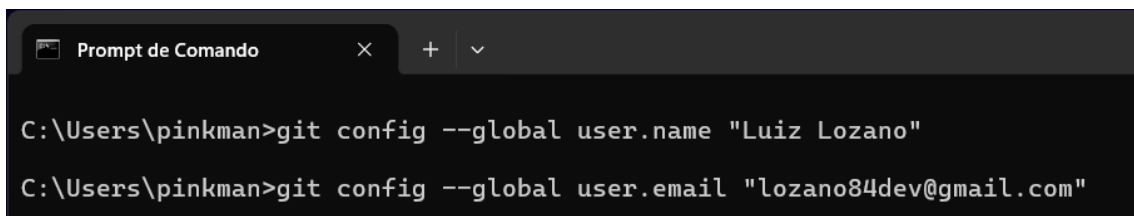
Aplicações Web

Professor Lozano

Após selecionar main (como nome padrão para o branch principal dos seus repositórios, hoje o padrão recomendado pela comunidade, substituindo o antigo "master"), avance até concluir a instalação.

Ao concluir, abra o terminal e vamos configurar o Git com seu nome de usuário e e-mail, essas informações ficam associadas a cada commit que você fizer, funcionando como sua "assinatura" no histórico do projeto:

- `git config --global user.name "Seu Nome"`
- `git config --global user.email "seuemail@exemplo.com"`

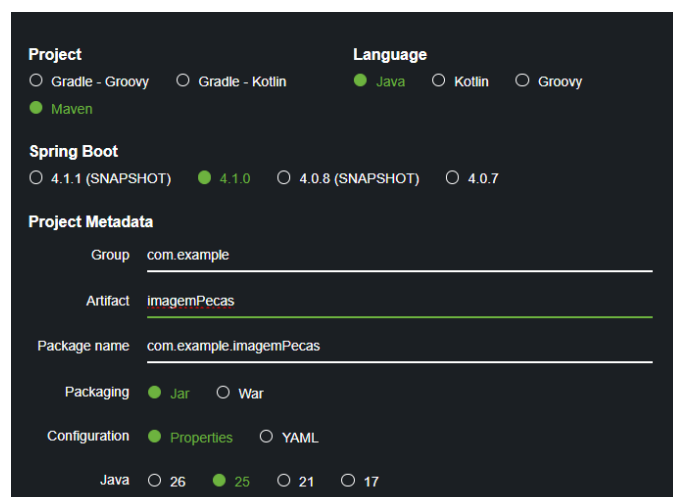


```
Prompt de Comando
C:\Users\pinkman>git config --global user.name "Luiz Lozano"
C:\Users\pinkman>git config --global user.email "lozano84dev@gmail.com"
```

Passo 8 – Criação do projeto da API com Spring Initializr.

Com todas as ferramentas devidamente instaladas, chegou o momento que estávamos esperando: começar a construir nossa API.

Para isso, vamos utilizar o Spring Initializr, uma ferramenta oficial do ecossistema Spring que gera a estrutura inicial do nosso projeto de forma rápida e organizada, sem precisarmos montar manualmente pastas, arquivos de configuração e dependências do zero. Acesse start.spring.io e vamos configurar a nossa API:



The image shows the Spring Initializr web form with the following configuration:

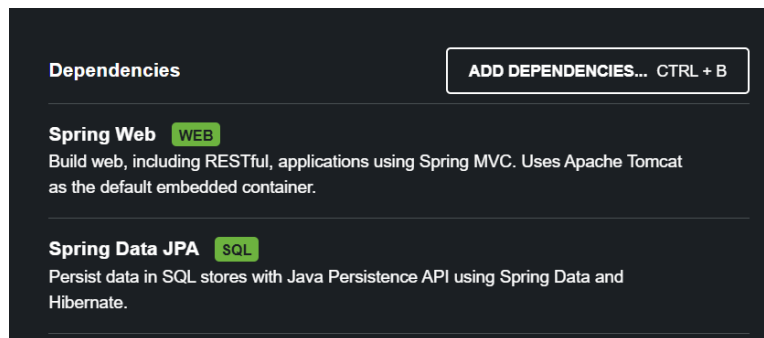
- Project:** ☒ Maven
- Language:** ☒ Java
- Spring Boot:** ☒ 4.1.0
- Project Metadata:**
 - Group: `com.example`
 - Artifact: `imagemPecas`
 - Package name: `com.example.imagemPecas`
- Packaging:** ☒ Jar
- Configuration:** ☒ Properties
- Java:** ☒ 25

Aplicações Web

Professor Lozano

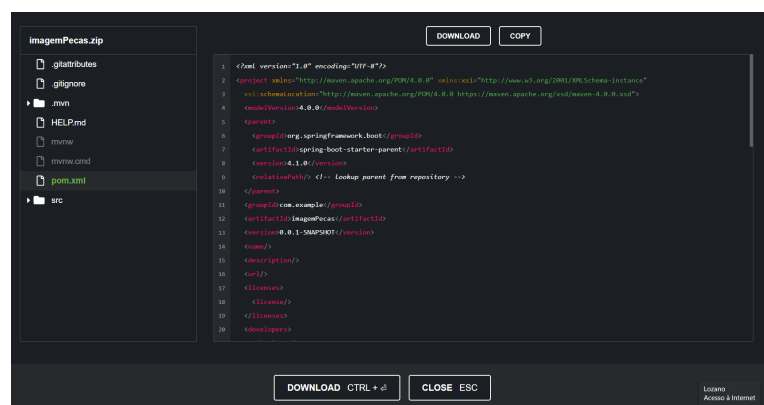
Agora vamos selecionar as dependências do nosso projeto, são elas que vão definir quais bibliotecas e funcionalidades o Spring Boot já vai trazer prontas pra gente, sem precisarmos configurar manualmente depois. Para o nosso projeto, vamos escolher duas:

- Spring Web: essa dependência é o que vai nos permitir construir a nossa API REST, ela já traz tudo que precisamos para criar rotas (endpoints), receber requisições HTTP e devolver respostas, sem precisarmos configurar um servidor web do zero.
- Spring Data JPA: essa aqui vai facilitar demais nossa vida na hora de conversar com o banco de dados. Com ela, conseguimos manipular informações do banco através de código Java (usando o padrão ORM — Object-Relational Mapping), sem precisarmos escrever SQL manualmente para cada operação básica.



Após adicionar as duas dependências, vamos clicar em Explore para dar uma olhada na estrutura do projeto que o Spring Initializr já vai gerar automaticamente pra gente, assim conseguimos entender como as pastas e arquivos ficam organizados antes mesmo de baixar o projeto.

Depois dessa conferida, clicamos em Download, para baixarmos o projeto já configurado com todas as opções e dependências que selecionamos.



Aplicações Web

Professor Lozano

Passo 9 – Versionando nosso projeto com Git e Github.

Com o esqueleto da nossa API já criado, chegou a hora de colocar nosso projeto sob controle de versão. Nesse passo, vamos fazer todo o fluxo básico do Git na prática: criar um repositório remoto no GitHub, clonar para nossa máquina, inserir os arquivos do projeto gerado pelo Spring Initializr, e realizar nosso primeiro commit e push.

Esse fluxo vai se repetir bastante ao longo do curso, então vale prestar atenção em cada etapa, é a base de como vamos versionar e entregar nosso código daqui pra frente.

Para começar, acesse sua conta no GitHub e crie um repositório público:

The screenshot shows the GitHub 'Create a new repository' interface. It is divided into three main sections: General, Configuration, and a Copilot optional section. In the General section, the repository name 'imagemPecas' is entered and validated as available. The Configuration section shows 'Public' visibility, 'Add README' turned off, 'Add .gitignore' set to 'No .gitignore', and 'Add license' set to 'No license'. The bottom section prompts the user to describe what they want Copilot to build, with a text area and a character count (0 / 30000). A green 'Create repository' button is at the bottom right.

Create a new repository
Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner * lcmlozano / Repository name * imagemPecas
✓ imagemPecas is available.

Great repository names are short and memorable. How about [fictional-engine](#)?

Description
0 / 350 characters

2 Configuration

Choose visibility * Public
Choose who can see and commit to this repository

Add README Off
READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore No .gitignore
.gitignore tells git which files not to track. [About ignoring files](#)

Add license No license
Licenses explain how others can use your code. [About licenses](#)

3 Jumpstart your project with Copilot (optional)

Tell [Copilot cloud agent](#) what you want to build in this repository. After creation, Copilot will open a pull request with generated files - such as a basic app, starter code, or other features you describe - then request your review when it's ready.

Prompt
Describe what you want Copilot to build
0 / 30000 characters

[Create repository](#)

Aplicações Web

Professor Lozano

Em nossa máquina, vamos escolher uma pasta, e vamos fazer o clone desse repositório. Para isso precisamos copiar o endereço do repositório (aquele que tem o .git no final) e utilizar o comando git clone.

Para isso acesse o terminal, vá até o diretório raiz (c:/). Crie uma pasta chamada projetos (utilizando o comando mkdir). Acesse a pasta projetos (utilizando cd projetos) e faça o clone do repositório que foi criado no github.

Agora, na nossa máquina, vamos escolher onde esse projeto vai morar localmente e fazer o clone do repositório que acabamos de criar.

Primeiro, precisamos copiar o endereço do repositório, aquele link que termina em .git. É esse endereço que o comando git clone vai usar para saber de onde baixar o projeto.

Vamos organizar isso direitinho: abra o terminal e siga os passos abaixo.

1. Vá até o diretório raiz do seu disco:

```
cd c:/
```

2. Crie uma pasta chamada projetos, que vai centralizar os projetos da nossa disciplina:

```
mkdir projetos
```

3. Acesse a pasta recém-criada:

```
cd projetos
```

4. Agora sim, faça o clone do repositório criado no GitHub (substituindo pelo endereço que você copiou):

```
git clone [endereço-do-repositório].git
```

Esse comando vai baixar (na verdade, criar uma cópia local completa, com histórico incluso) o repositório remoto dentro da pasta projetos, deixando tudo pronto para recebermos os arquivos da nossa API.

Aplicações Web

Professor Lozano

```
Prompt de Comando

C:\>mkdir projetos

C:\>cd projetos

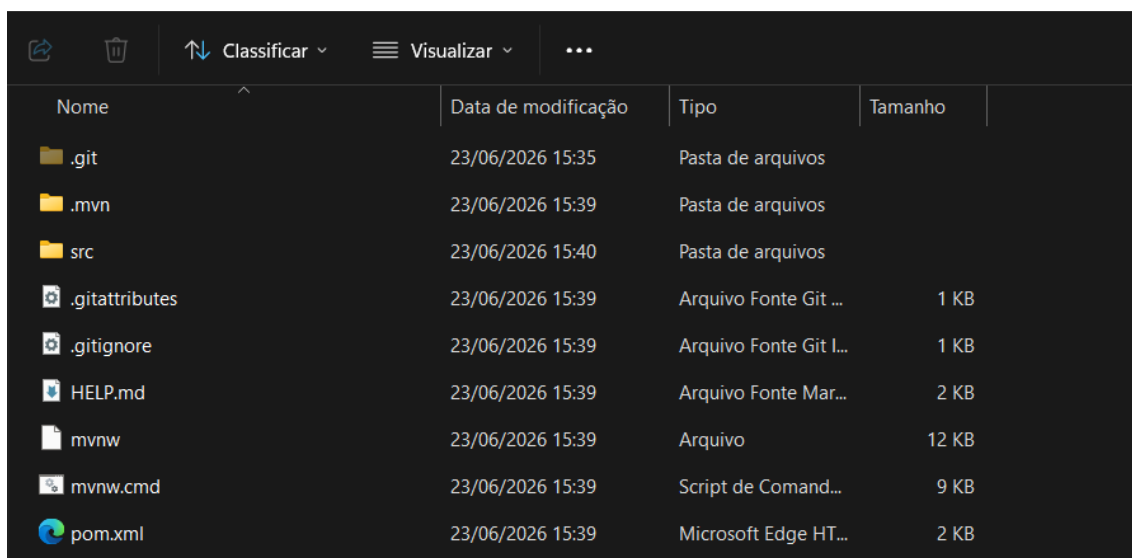
C:\projetos>git clone https://github.com/lcmlozano/imagemPecas.git
Cloning into 'imagemPecas'...
warning: You appear to have cloned an empty repository.

C:\projetos>dir
O volume na unidade C não tem nome.
O Número de Série do Volume é 4483-289A

Pasta de C:\projetos

14/07/2026  21:11    <DIR>          .
14/07/2026  21:11    <DIR>          imagemPecas
                0 arquivo(s)          0 bytes
                2 pasta(s)  288.366.505.984 bytes disponíveis
```

Agora vamos descompactar o arquivo imagemPecas.zip dentro da pasta raiz do projeto que acabamos de clonar:



Nome	Data de modificação	Tipo	Tamanho
.git	23/06/2026 15:35	Pasta de arquivos	
.mvn	23/06/2026 15:39	Pasta de arquivos	
src	23/06/2026 15:40	Pasta de arquivos	
.gitattributes	23/06/2026 15:39	Arquivo Fonte Git ...	1 KB
.gitignore	23/06/2026 15:39	Arquivo Fonte Git I...	1 KB
HELP.md	23/06/2026 15:39	Arquivo Fonte Mar...	2 KB
mvnw	23/06/2026 15:39	Arquivo	12 KB
mvnw.cmd	23/06/2026 15:39	Script de Comand...	9 KB
pom.xml	23/06/2026 15:39	Microsoft Edge HT...	2 KB

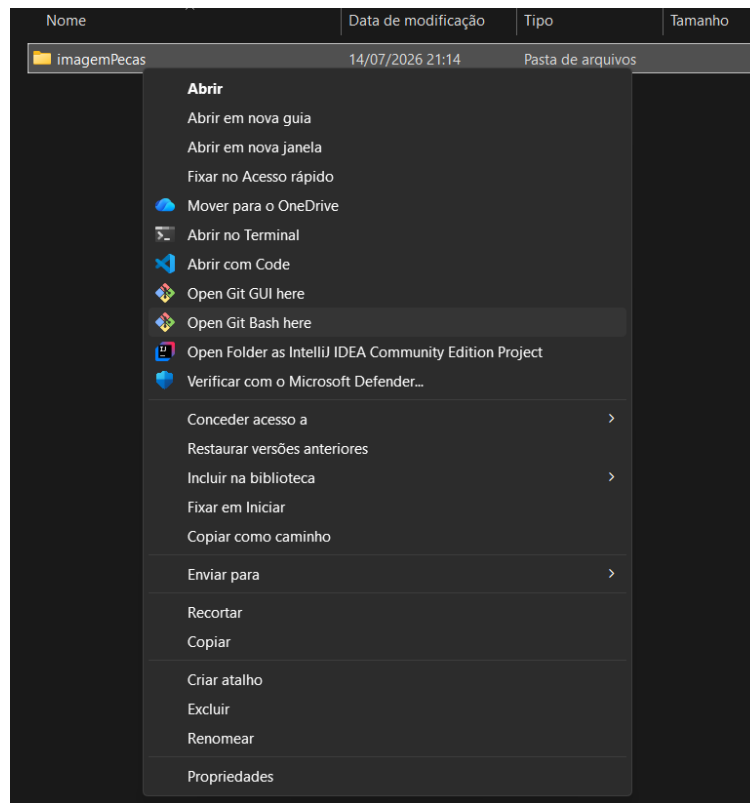
Obs.: antes de copiar os arquivos, certifique-se de que a visualização de arquivos ocultos esteja ativada no seu sistema. Isso é importante porque a pasta .git (criada pelo clone) fica oculta por padrão, e precisamos enxergá-la para garantir que estamos trabalhando na pasta certa.

Aplicações Web

Professor Lozano

Outro ponto de atenção: você deve copiar os arquivos de dentro da pasta descompactada, e não a pasta em si. Se copiar a pasta inteira, vamos acabar com uma pasta dentro da outra, com o mesmo nome, o que vai bagunçar a estrutura do nosso projeto.

Com os arquivos copiados corretamente para a pasta raiz do projeto, selecione essa pasta, clique com o botão direito do mouse e escolha a opção "Open Git Bash Here":



Com o terminal aberto diretamente na pasta do projeto, digite o comando: git status.

```
MINGW64/c/projetos/imagemPecas
pinkman@pinkman MINGW64 /c/projetos/imagemPecas (main)
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitattributes
        .gitignore
        .mvn/
        mvnw
        mvnw.cmd
        pom.xml
        src/

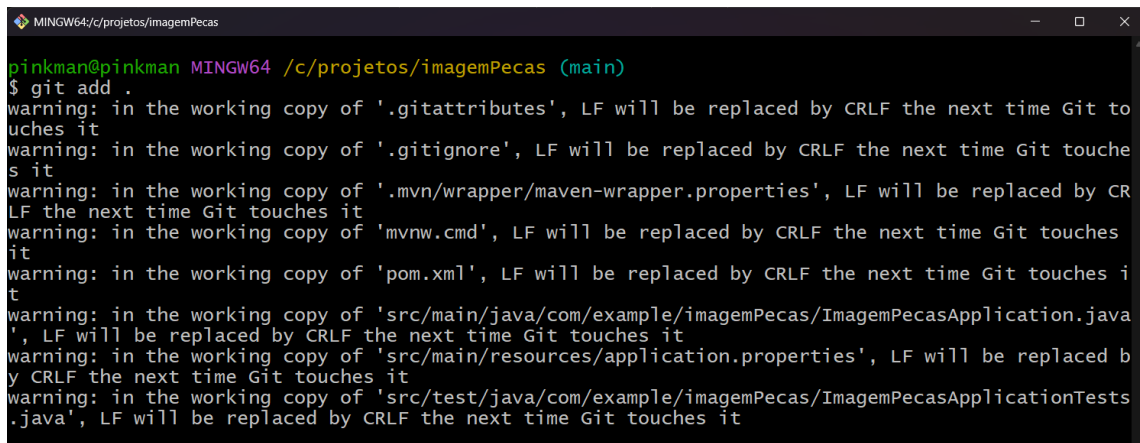
nothing added to commit but untracked files present (use "git add" to track)
```

Aplicações Web

Professor Lozano

Digite o comando: `git add .`

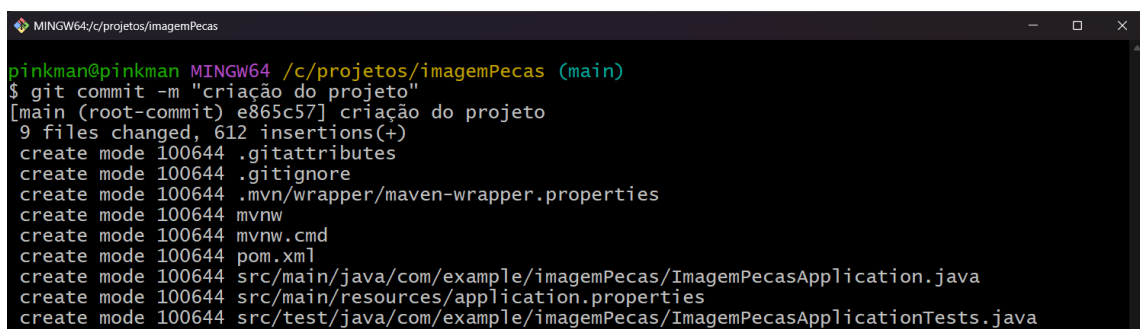
Esse comando adiciona todas as alterações da pasta atual (o `.` representa "aqui, e tudo que estiver dentro") à chamada staging área, uma espécie de "área de preparação" do Git, onde ficam os arquivos que serão incluídos no próximo commit. Ou seja, ainda não commitamos nada, só sinalizamos pro Git: "essas são as mudanças que eu quero registrar".



```
MINGW64/c:/projetos/imagemPecas
pinkman@pinkman MINGW64 /c:/projetos/imagemPecas (main)
$ git add .
warning: in the working copy of '.gitattributes', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of '.mvn/wrapper/maven-wrapper.properties', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'mvnw.cmd', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'pom.xml', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/main/java/com/example/imagemPecas/ImagemPecasApplication.java', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/main/resources/application.properties', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/test/java/com/example/imagemPecas/ImagemPecasApplicationTests.java', LF will be replaced by CRLF the next time Git touches it
```

Agora digite o comando: `git commit -m "criação do projeto"`

Esse comando finalmente registra as alterações que colocamos na staging area, criando um commit, ou seja, um "retrato" (snapshot) do estado atual do projeto naquele momento, junto com uma mensagem descritiva (-m) explicando o que foi feito. Esse é o nosso primeiro commit do projeto, marcando oficialmente o início do histórico de versões da nossa API.



```
MINGW64/c:/projetos/imagemPecas
pinkman@pinkman MINGW64 /c:/projetos/imagemPecas (main)
$ git commit -m "criação do projeto"
[main (root-commit) e865c57] criação do projeto
9 files changed, 612 insertions(+)
create mode 100644 .gitattributes
create mode 100644 .gitignore
create mode 100644 .mvn/wrapper/maven-wrapper.properties
create mode 100644 mvnw
create mode 100644 mvnw.cmd
create mode 100644 pom.xml
create mode 100644 src/main/java/com/example/imagemPecas/ImagemPecasApplication.java
create mode 100644 src/main/resources/application.properties
create mode 100644 src/test/java/com/example/imagemPecas/ImagemPecasApplicationTests.java
```

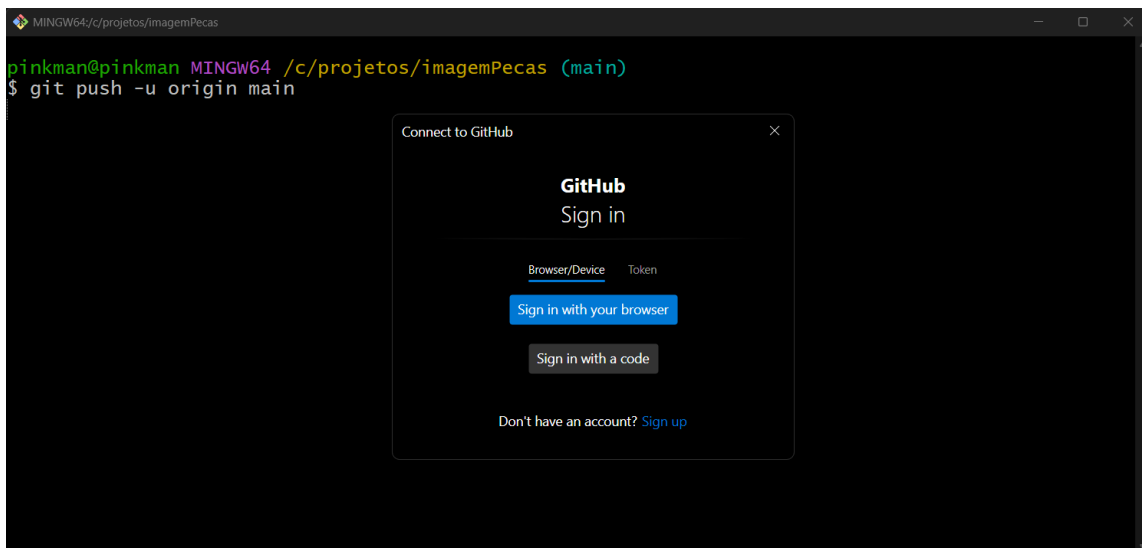
Aplicações Web

Professor Lozano

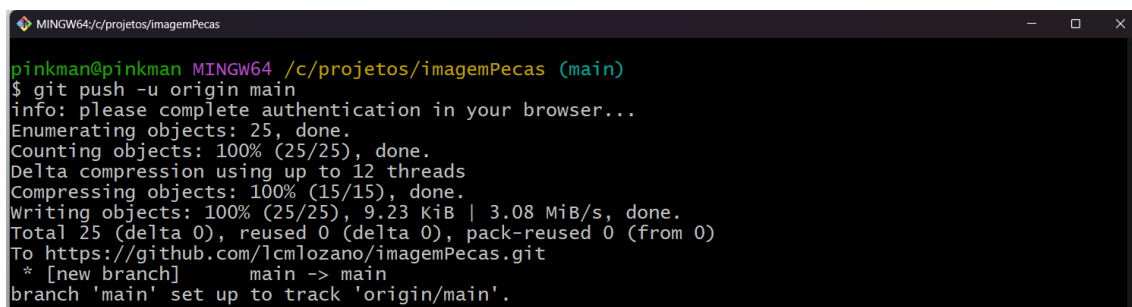
Por fim, digite o comando: `git push -u origin main`

Esse comando envia (faz o "push") todos os commits que criamos localmente para o repositório remoto no GitHub, é a partir daqui que nosso código realmente passa a existir na nuvem, e não só na nossa máquina. A flag `-u origin main` cria um vínculo entre nossa branch local `main` e a branch `main` do repositório remoto (`origin`), fazendo com que, nos próximos pushes, não precisemos mais especificar isso, bastará digitar `git push`.

Nesse momento, é provável que o GitHub abra uma tela de login para autenticação. Recomendo que essa autenticação seja feita via navegador, ou seja, é importante que você já esteja logado no GitHub no seu browser, para que o processo seja concluído de forma rápida e automática, sem precisar digitar usuário e senha manualmente no terminal.



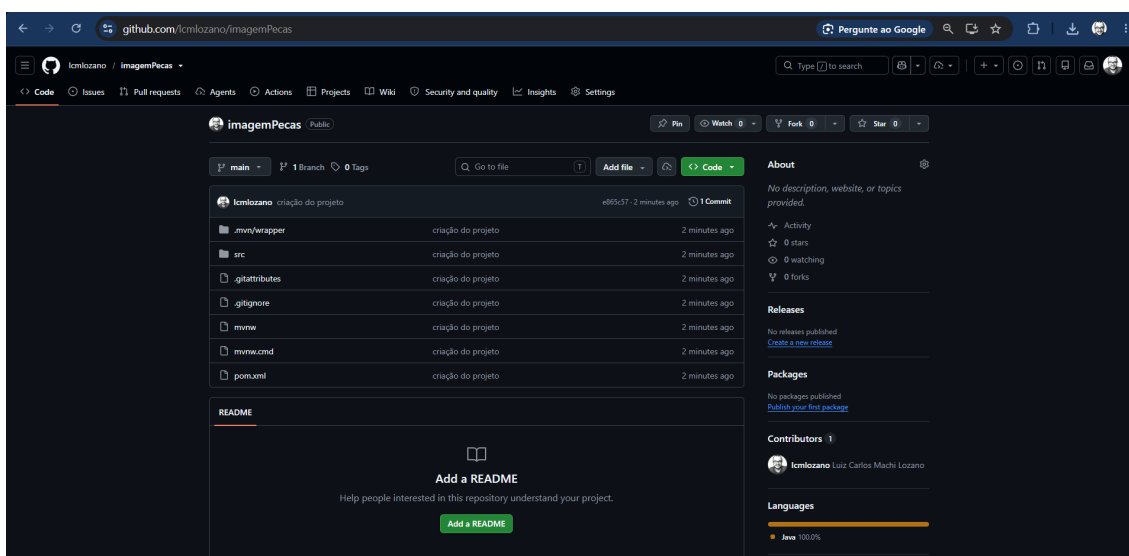
Ao concluir o login, os arquivos serão automaticamente carregados para o servidor remoto, e pronto, nosso projeto já está versionado e disponível no GitHub.



Aplicações Web

Professor Lozano

Ao acessar o repositório no GitHub pelo navegador, vamos conferir que a última versão do nosso projeto já está lá, disponível na nuvem, confirmando que todo o fluxo de versionamento que acabamos de praticar (criar repositório, clonar, adicionar arquivos, commit e push) funcionou de ponta a ponta.



A partir de agora, nosso projeto já está vinculado ao GitHub, ou seja, esse vínculo entre repositório local e remoto que acabamos de configurar (`git push -u origin main`) vai continuar valendo para o resto do curso. Periodicamente, conforme formos avançando no desenvolvimento, faremos novos commits e pushes para manter esse histórico sempre atualizado.

Passo 10 – Abrindo o projeto no IntelliJ.

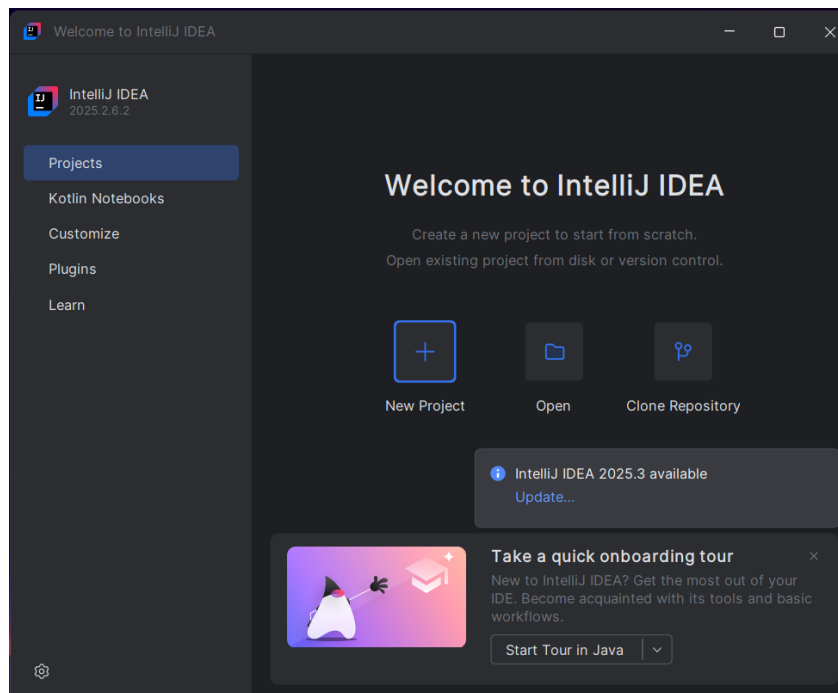
Com o projeto já criado, versionado e disponível no GitHub, chegou a hora de abri-lo no IntelliJ IDEA para começarmos a trabalhar no código.

Abra o IntelliJ, vá até a opção Open e navegue até a pasta do projeto que clonamos anteriormente.

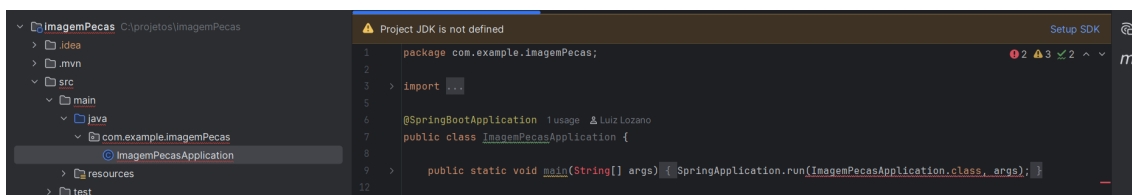
Ao abrir, o IntelliJ pode exibir um aviso perguntando se confiamos no projeto, isso é uma medida de segurança da própria IDE, já que projetos podem conter scripts de build que são executados automaticamente. Como o projeto é o nosso mesmo, criado por nós no Spring Initializr, clique em Trust Project:

Aplicações Web

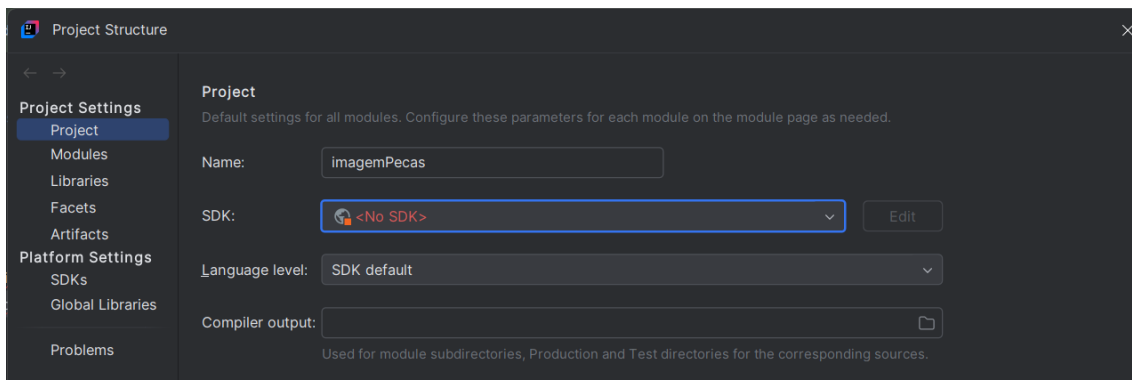
Professor Lozano



Ao abrir o projeto, se navegarmos até `src / main / ... / ImagemPecasApplication`, vamos notar que a classe principal está apresentando um erro. Isso acontece porque o IntelliJ ainda não está reconhecendo o JDK que instalamos lá no Passo 1:



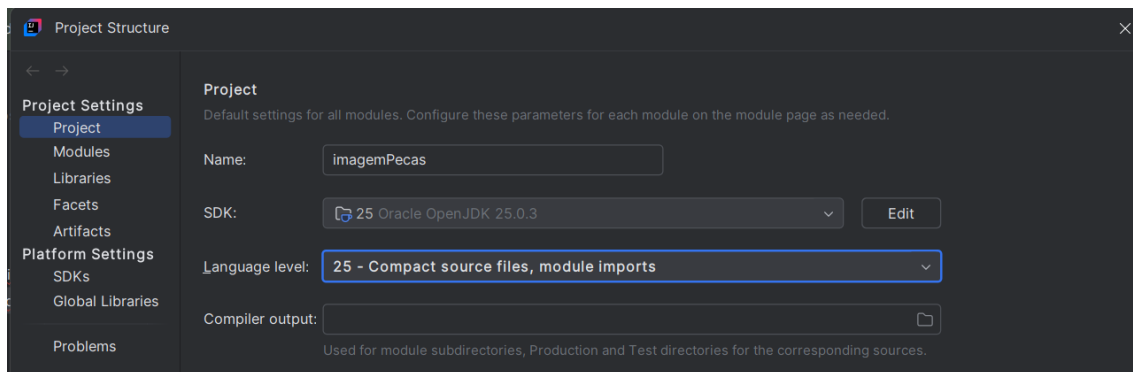
Para resolver esse problema, vamos até `File → Project Structure`. Na aba `Project`, vamos notar que o campo de SDK está vazio, ou seja, o IntelliJ simplesmente não sabe ainda qual versão do Java deve usar para compilar e rodar nosso projeto:



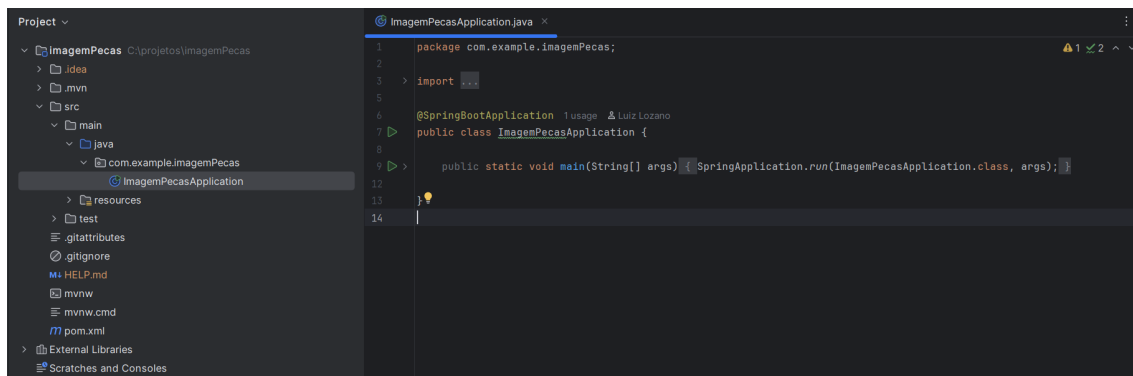
Aplicações Web

Professor Lozano

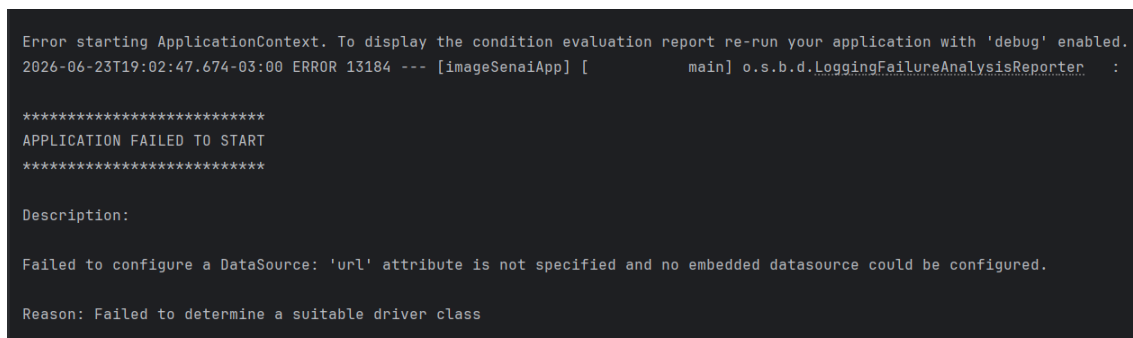
Selecione a opção 25 (referente ao JDK 25 que instalamos no Passo 1) e clique em Apply:



Após aplicar a alteração, podemos notar que o problema foi resolvido, o IntelliJ agora reconhece corretamente o JDK 25, e o erro que aparecia na classe ImagemPecasApplication desaparece:



Agora vamos executar nossa classe principal (ImagemPecasApplication). O Spring Boot será inicializado normalmente, porém, ao final, vamos nos deparar com o seguinte erro:



Aplicações Web

Professor Lozano

Esse erro ocorre porque, ao adicionarmos a dependência Spring Data JPA lá no Passo 8, o Spring passou a esperar uma conexão configurada com um banco de dados (um DataSource) assim que a aplicação sobe, e ainda não configuramos nenhum banco, já que isso só vai acontecer quando conectarmos nossa API a um banco de dados de verdade.

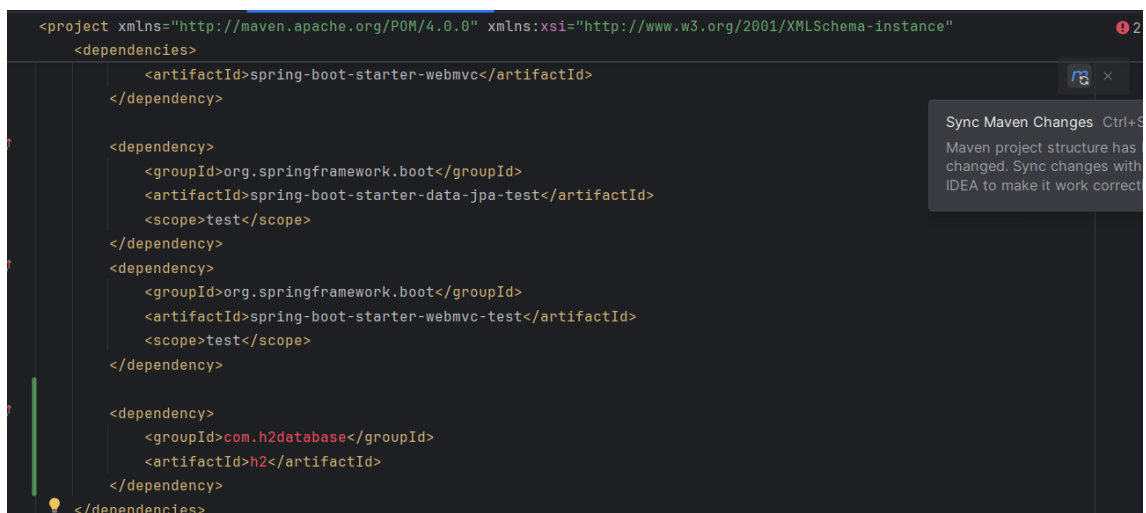
Por enquanto, para contornar esse erro e conseguirmos rodar a aplicação normalmente, vamos até o arquivo pom.xml e inserir a seguinte dependência:

```
<dependency>

    <groupId>com.h2database</groupId>

    <artifactId>h2</artifactId>

</dependency>
```



Nós inserimos a dependência do banco H2, um banco de dados leve, que roda em memória, muito utilizado justamente para esses cenários de desenvolvimento e testes rápidos, sem precisar de um banco "de verdade" instalado na máquina. Ele resolve nosso erro atual porque fornece automaticamente um DataSource temporário para o Spring usar, enquanto não configuramos nossa conexão definitiva.

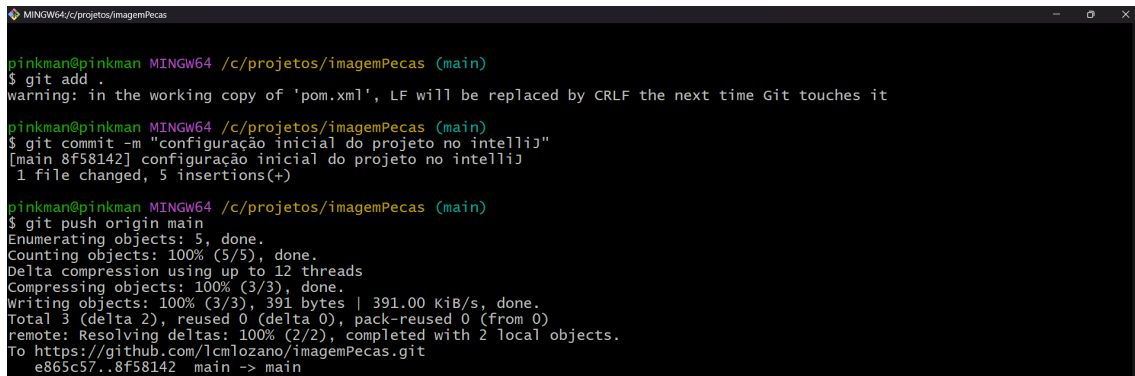
Após inserir a dependência do H2 no pom.xml, o Maven precisa ser avisado de que há uma nova dependência para baixar e gerenciar. Para isso, clique no ícone do Maven que aparece no canto superior direito do IntelliJ, isso vai disparar o download da dependência e atualizar o projeto.

Com esse processo concluído, podemos executar novamente nossa classe principal. Dessa vez, o projeto Spring vai subir sem nenhum erro, já que o H2 fornece o DataSource que faltava.

Aplicações Web

Professor Lozano

Para fechar o passo, vamos repetir o fluxo que já praticamos no Passo 9: fazer o commit e subir essa última versão do projeto para o GitHub.



```
pinkman@pinkman MINGW64 /c/projetos/imagemPecas (main)
$ git add .
warning: in the working copy of 'pom.xml', LF will be replaced by CRLF the next time Git touches it
pinkman@pinkman MINGW64 /c/projetos/imagemPecas (main)
$ git commit -m "configuração inicial do projeto no intelliJ"
[main 8f58142] configuração inicial do projeto no intelliJ
1 file changed, 5 insertions(+)
pinkman@pinkman MINGW64 /c/projetos/imagemPecas (main)
$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 391 bytes | 391.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/lcmlozano/imagemPecas.git
e865c57..8f58142  main -> main
```